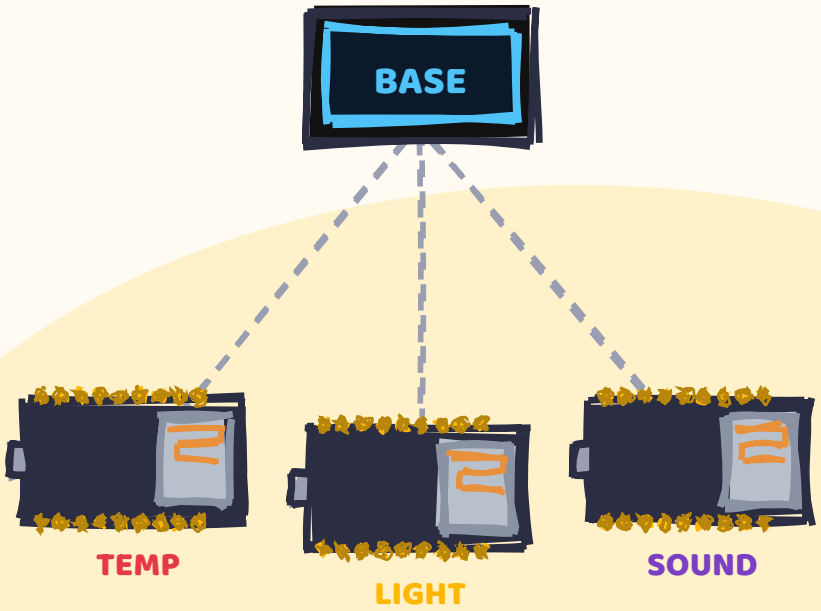
Tech Leaders · Class 5

ESP-NOW Wireless Sensor Network

Three nodes, one base, no router needed

Packets & addresses **Network topology** **Packet loss %**



Did you know?
Smart buildings can have thousands of small wireless sensors reporting to a few hubs, saving energy by heating only rooms in use.

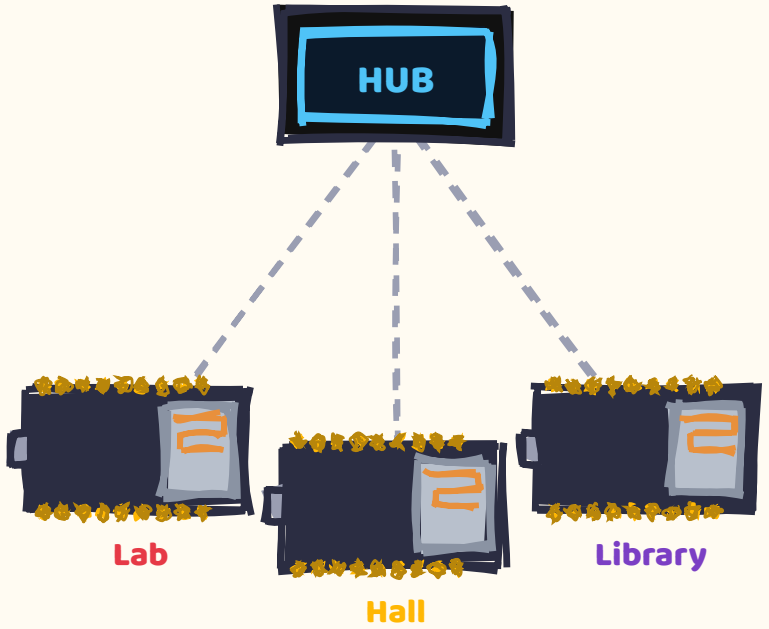
LEARN

The engineering problem

Wires everywhere? No thanks.

Our school wants to know

Which rooms are too hot, too dark or too noisy for learning? Running cables to every room is expensive and messy. We need sensors that **talk wirelessly** to one hub, and we need to know **when a message goes missing**.



Wi-Fi?

Bluetooth?

Talk directly!

Today: you build a 3-node sensor network, then **measure how reliable it is** like a real network engineer.

Learning objectives

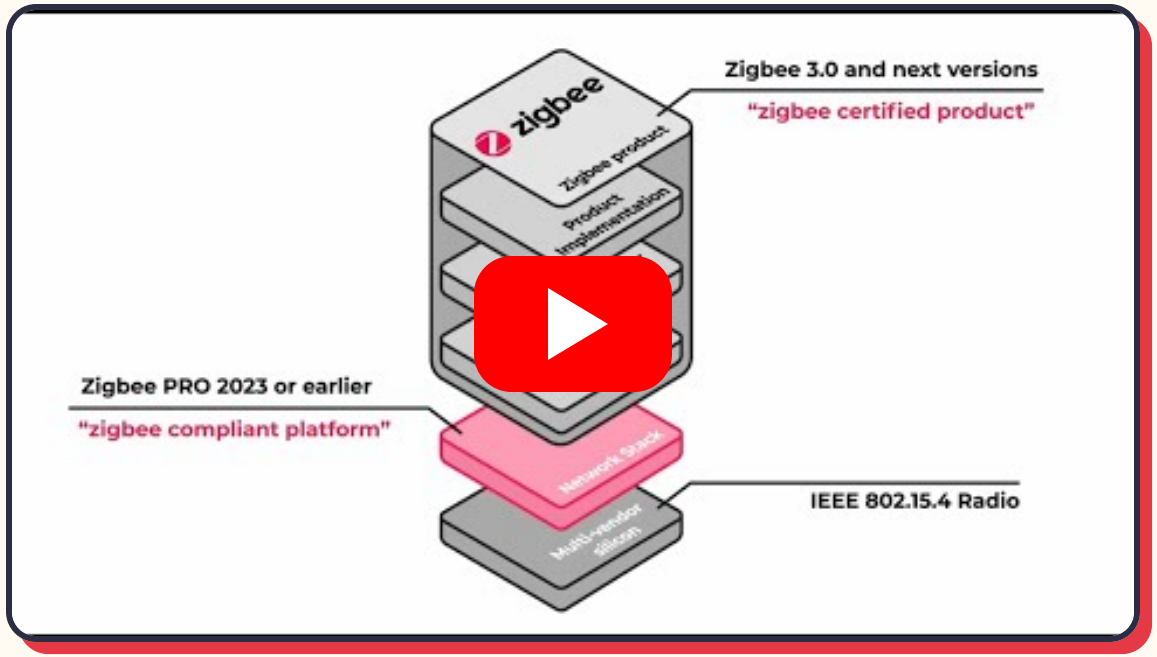
By the end of this class I can...

1	**Packets:** explain how data travels as **packets** with an address, a payload and a sequence number	**Proof:** label the 3 parts of our message on a diagram
2	**Addresses:** find a board's **MAC address** and send data to one specific device	**Proof:** your base receives from your node, not the next table's
3	**Topology:** compare **star and mesh** networks and justify the right one for a building	**Proof:** draw our network and name its single point of failure
4	**Code:** write sender and receiver code using a **struct** and a **callback**	**Proof:** all 3 nodes appear live on the base OLED
5	**Reliability:** measure **packet loss** as a percentage and detect an **offline** node	**Proof:** a loss table at 3 distances and OFFLINE shown on the OLED

LEARN

Watch and think

Video time!



What is Zigbee?

Connectivity Standards Alliance · about 4 minutes · youtu.be/2p8G6JMvgUw

Watch for...

1. What is a **mesh** network?
2. Why do smart home devices use low-power radio?
3. What happens if one device in a mesh switches off?

Scan to watch

After the video: Compare: is our ESP-NOW network a star or a mesh? Which would you use for a whole school?

Objective 1 · Packets

Data travels in packets

The idea

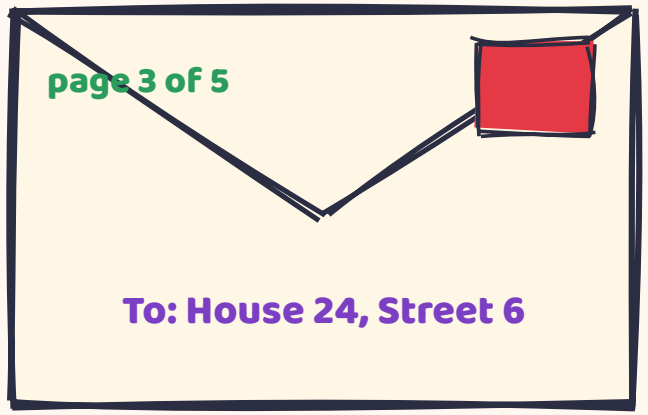
Networks never send a 'stream of meaning'. They send small **packets**. Each one has:

1. **Address**: who it is for
2. **Payload**: the actual data
3. **Sequence number**: which packet this is, so missing ones can be spotted

Size limit

An ESP-NOW payload can be at most 250 bytes. Our message is only 9 bytes.

Example: a posted letter



The house address is the **MAC address**. The letter inside is the **payload** (24.5 °C). 'Page 3 of 5' is the **sequence number**: if page 4 never arrives, you know it got lost.

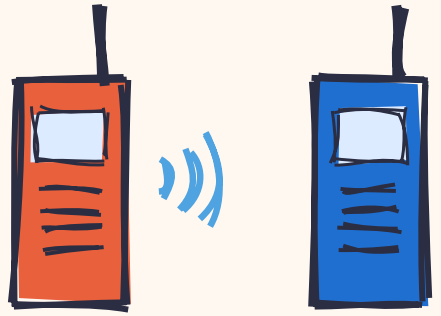
LEARN

Objective 1 · Packets · protocols

A protocol is an agreed set of rules

	Wi-Fi	Bluetooth	ESP-NOW
Needs a router?	Yes	No	No
Range (open air)	~50 m	~10 m	100 m+
Set-up time	Seconds	Pairing	Instant
Best for	Internet	Phones, earbuds	Board to board

Example: walkie-talkies



A phone call needs a tower in the middle. A walkie-talkie just presses and talks to the other radio. **ESP-NOW is the walkie-talkie**: fast, direct, no router.

Trade-off

ESP-NOW has no internet. For the internet we use Wi-Fi + MQTT next class (TL-06).

Objective 2 · Addresses

Every board has a unique MAC address

The idea

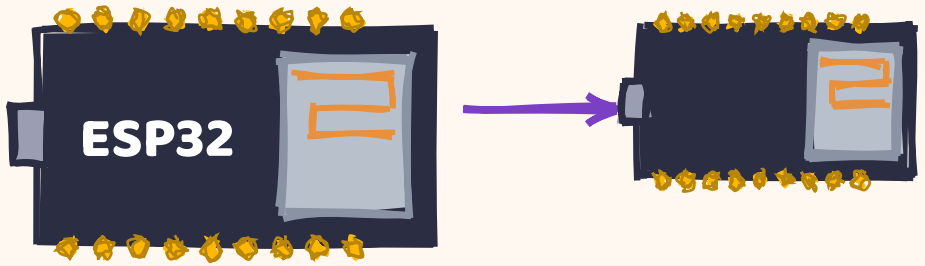
A **MAC address** is 6 bytes burned into the chip at the factory, written in hex: **24:6F:28:A1:B2:C3**. In ESP-NOW the sender must add the receiver as a **peer** using its MAC.

Unicast = to one MAC. **Broadcast** = to FF:FF:FF:FF:FF:FF, everyone hears it.

Lab habit

Write your base's MAC on masking tape and stick it on the board. You will need it in every sender.

Example: calling a name in the hall



24:6F:28:A1:B2:C3

Shouting "Everyone!" in the school hall is a **broadcast**. Saying "Ayesha, Class 10B" is **unicast**. In a lab with 8 groups, broadcast means your base would show everyone's data.

Objective 3 · Topology

Star topology: everyone reports to one hub

The idea

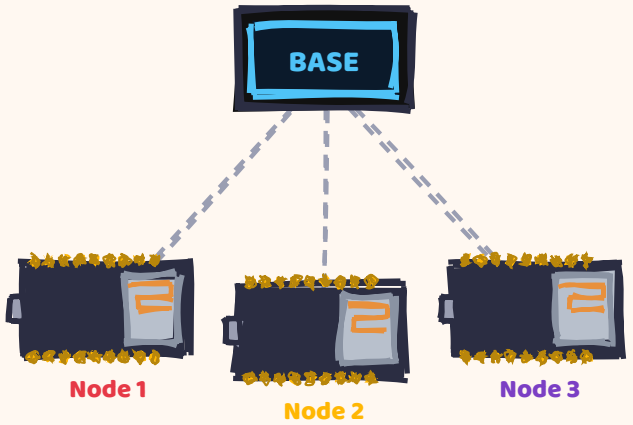
In a **star**, every node talks only to the central hub. It is simple to build and easy to debug.

The weakness: the hub is a **single point of failure**. If it stops, the whole network stops. Nodes that are out of range of the hub cannot join.

Think

Our network is a star. It is right for one floor of a school. Is it right for a whole campus?

Example: handing in homework



Every student hands work straight to the teacher. Easy to track who has submitted. But if the teacher is absent, **nothing gets collected**. That is our base station.

LEARN

Objective 3 · Topology

Mesh topology: nodes pass messages along

The idea

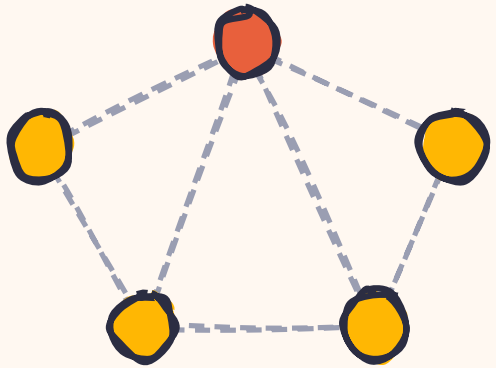
In a **mesh**, nodes can relay messages for each other. If one path breaks, the message finds another route. Range grows with every node added.

The cost: more complex code, more radio traffic and harder debugging.

Engineer's rule

Choose star when nodes are close and simple matters. Choose mesh when distance or reliability matters most.

Example: smart light bulbs



Zigbee and Thread smart bulbs form a mesh. A bulb in the garden is too far from the hub, but the bulb in the kitchen **passes its message on**. Remove one bulb and the others re-route.

Objective 4 · Code

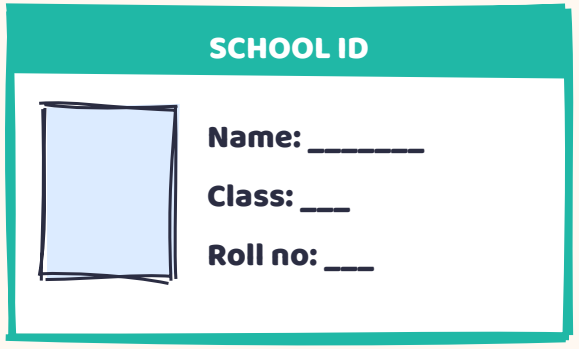
A struct = one fixed-layout message

The idea

A **struct** groups several values into one named package with a fixed layout. Sender and receiver must use the **exact same struct**, or the bytes are read wrongly.

```
typedef struct {
  uint8_t id;   // which node
  float value;  // the reading
  uint32_t seq; // packet number
} Msg;
```

Example: a school ID card



Every ID card has the **same boxes in the same places**: name, class, roll number. A guard can read any card instantly. If one school printed the roll number where the name should be, it would be read wrong. Same with a struct.

LEARN

Objective 4 · Code

A callback runs when a message arrives

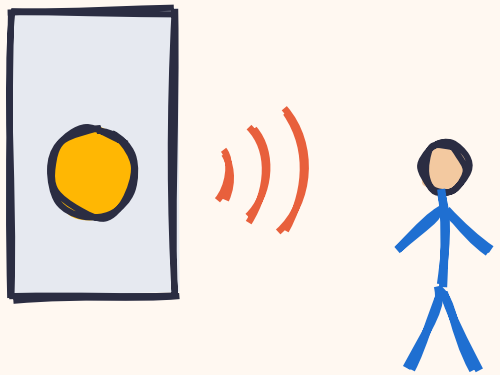
The idea

We do not keep checking "has a message come yet?" in **loop()**. We give ESP-NOW a function, and it **calls us back** the moment a packet arrives:

```
esp_now_register_recv_cb(onRecv);
```

Keep callbacks **short**: copy the data and leave. Do drawing and printing in loop().

Example: a doorbell



You do not stand at the front door all day waiting for a visitor. You carry on with your work and the **bell interrupts you** when someone arrives. You answer quickly and go back to work.

LEARN

Objective 5 · Reliability

Packet loss: what went missing?

The idea

Each node counts up its **seq** number. The base knows how many were **sent** (last seq - first seq + 1) and how many it **received**.

Loss % = (sent - received) ÷ sent × 100

Walls, distance, metal and other Wi-Fi all increase loss.

Example: a worked calculation



Node 2 started at seq 101 and the last packet was seq 300. So **200 were sent**. The base counted **188**.
Loss = (200 - 188) ÷ 200 × 100 = **6%**.

Context matters

Under 5% loss is fine for a temperature sensor. Is it fine for a fire alarm?

Objective 5 · Reliability

No news is bad news: offline nodes

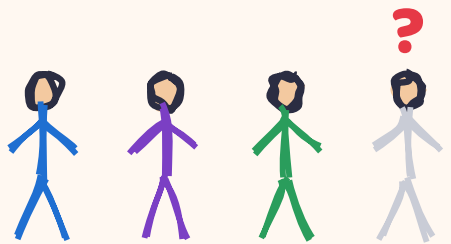
The idea

A node that dies sends **nothing**, so there is no error message. The base must notice the **silence**.

Every node sends regularly (a **heartbeat**). The base records the time of the last packet. If more than **10 seconds** pass, it shows **OFFLINE**.

```
if (millis() - last[i] > 10000)
```

Example: a hiking group check-in



A school hiking group agrees: every team radios in every 10 minutes. If one team misses **two check-ins**, the leader starts a search. Nobody waits for the lost team to call and say "we are lost".

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Kit per network (4 groups share)
2. Find your base's MAC address
3. Wire the three sensor nodes
4. Wire the base station display
5. The sender (same code on all 3 nodes)
6. The base: receive with a callback
7. Show readings, loss % and OFFLINE

MAKE

Build it

Kit per network (4 groups share)

4 × ESP32
3 nodes + 1 base, USB cables

DHT22
Node 1: temperature

LDR module
Node 2: light level

KY-037 sound
Node 3: noise level

OLED 0.96"
Base: live summary

4 breadboards
+ jumpers, power banks

Software: Arduino IDE 2, ESP32 board package (core 3.x), libraries: **DHT sensor library**, **Adafruit SSD1306**, **Adafruit GFX**. ESP-NOW is built in.

Team roles
Network lead (base) · 3 node engineers · 1 data analyst who runs the range test.

MAKE

Objective 2 · Build step 1 of 4

Find your base's MAC address

```
#include <WiFi.h>
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);
  delay(100);
  Serial.println(WiFi.macAddress());
}
void loop() {}
```

a Upload this to the board that will be the **base**.

b Open Serial Monitor at **115200**. Press EN (reset).

c Copy the 6 pairs, e.g. 24:6F:28:A1:B2:C3.

d Turn it into code: `{0x24,0x6F,0x28,0xA1,0xB2,0xC3}`

Check
Every group's MAC is different. If two match, someone copied!

00:00:00...? Wi-Fi was not started yet. Keep the delay(100) after WiFi.mode().

MAKE

Objective 4 · Build step 2 of 4

Wire the three sensor nodes

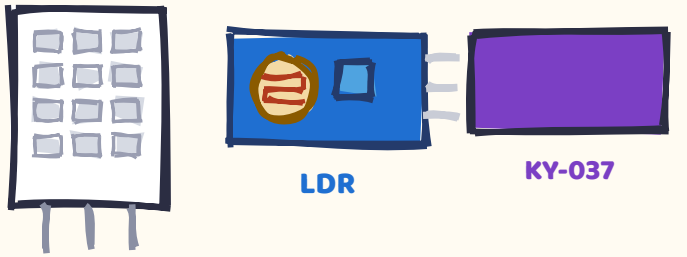
a **Node 1** DHT22: + to 3V3, - to GND, OUT to **GPIO 4** (most modules include the 10k pull-up).

b **Node 2** LDR module: VCC to 3V3, GND to GND, AO to **GPIO 34**.

c **Node 3** KY-037: + to 3V3, G to GND, AO to **GPIO 35**. Turn its blue screw until the LED just goes off in a quiet room.

d Use only **ADC1** pins (32 to 39) for analog. ADC2 pins stop working when the radio is on.

e Label each board with its **NODE_ID** (1, 2, 3) on masking tape.



Check: Each node powers on from a power bank and its LED lights.

Power sensors from **3V3**, not 5V: ESP32 pins are not 5 V tolerant.

MAKE

Objective 3 · Build step 3 of 4

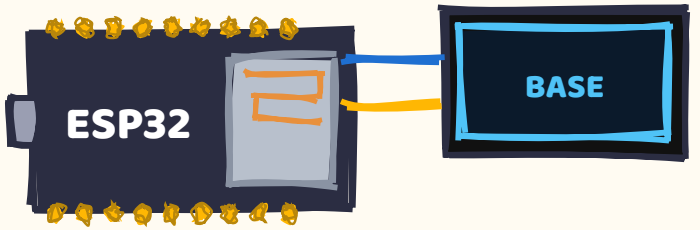
Wire the base station display

a OLED VCC to **3V3**, GND to GND.

b OLED SDA to **GPIO 21**, SCL to **GPIO 22** (the ESP32's default I2C pins).

c Upload a 'Hello' test from the SSD1306 examples. Address is usually **0x3C**.

d Keep the base near the middle of the room: in a star, the hub's position decides the range of the whole network.



Check: OLED shows text. If not, run an I2C scanner: no address = wiring problem.

MAKE

Objective 4 · Code part 1

The sender (same code on all 3 nodes)

```
#include <esp_now.h>
#include <WiFi.h>
#include <DHT.h>
#define NODE_ID 1   // 1 temp, 2 light, 3 sound
uint8_t BASE[] = {0x24,0x6F,0x28,0xA1,0xB2,0xC3};
typedef struct { uint8_t id; float value; uint32_t seq; } Msg;
Msg m;
DHT dht(4, DHT22);

float soundLevel() {          // peak-to-peak over 50 ms
  int lo = 4095, hi = 0; unsigned long t = millis();
  while (millis() - t < 50) { int v = analogRead(35);
    lo = min(lo, v); hi = max(hi, v); }
  return hi - lo;
}
void setup() {
  dht.begin();
  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_peer_info_t p = {};
  memcpy(p.peer_addr, BASE, 6);  // the base's MAC
  esp_now_add_peer(&p);
  m.id = NODE_ID;
}
void loop() {
  if (NODE_ID == 1) m.value = dht.readTemperature();
  if (NODE_ID == 2) m.value = analogRead(34);
  if (NODE_ID == 3) m.value = soundLevel();
  m.seq++;                                  // packet number
  esp_now_send(BASE, (uint8_t*)&m, sizeof(m));
  delay(2000);
}
```

O1 in code

BASE = address, **value** = payload, **seq** = sequence number. The whole packet is just 9 bytes.

O2 in code

esp_now_add_peer tells the radio who it may talk to. Wrong MAC = silence, no error.

Change one line

Only **NODE_ID** changes between nodes. One codebase, three devices: that is how real fleets are managed.

MAKE

Objective 4 · Code part 2

The base: receive with a callback

```
#include <esp_now.h>
#include <WiFi.h>
#include <Adafruit_SSD1306.h>
typedef struct { uint8_t id; float value; uint32_t seq; } Msg;
Adafruit_SSD1306 oled(128, 64, &Wire, -1);

volatile float val[4];
volatile uint32_t last[4], got[4], firstSeq[4], lastSeq[4];

// core 3.x signature (core 2.x uses: const uint8_t *mac)
void onRecv(const esp_now_recv_info_t *info,
            const uint8_t *data, int len) {
  if (len != sizeof(Msg)) return;     // not our struct
  Msg m; memcpy(&m, data, sizeof(m));
  if (m.id < 1 || m.id > 3) return;
  if (got[m.id] == 0) firstSeq[m.id] = m.seq;
  val[m.id] = m.value;
  lastSeq[m.id] = m.seq;
  got[m.id]++;
  last[m.id] = millis();              // heartbeat time
}
void setup() {
  WiFi.mode(WIFI_STA);
  esp_now_init();
  esp_now_register_recv_cb(onRecv);   // the doorbell
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.setTextColor(WHITE);
}
```

Short callback

It only **copies and counts**. No printing, no delay. Like answering the doorbell quickly.

Safety check

len != sizeof(Msg) rejects packets that are not ours. Never trust incoming data.

Ready for O5

firstSeq, lastSeq, got and **last** are everything we need to calculate loss and spot an offline node.

MAKE

Objective 5 · Code part 3

Show readings, loss % and OFFLINE

```
const char* NAME[] = {"", "Temp", "Light", "Sound"};

void loop() {
  oled.clearDisplay();
  oled.setCursor(0, 0);
  for (int i = 1; i <= 3; i++) {
    oled.print(NAME[i]); oled.print(": ");
    if (got[i] == 0 || millis() - last[i] > 10000) {
      oled.println("OFFLINE");        // silence = problem
      continue;
    }
    uint32_t sent = lastSeq[i] - firstSeq[i] + 1;
    float loss = 100.0 * (sent - got[i]) / sent;
    oled.print(val[i], 1);
    oled.print(" L:"); oled.print(loss, 0); oled.println("%");
  }
  oled.display();
  delay(500);
}
```

```
Temp: 27.4 L:1%
Light: 2310 L:3%
Sound: OFFLINE
```

Test it

Unplug node 3. Count: does OFFLINE appear after 10 s?

The maths from O5

sent = last seq - first seq + 1. Loss = (sent - received) ÷ sent × 100. The code is the formula from slide 11.

Why 10 seconds?

Nodes send every 2 s, so 10 s = 5 missed heartbeats. One lost packet must not trigger a false alarm.

Objective 5 · Build step 4 of 4

Experiment: how does distance affect loss?

Method

1. Reset the base so counters start at 0.
2. Place node 2 at **2 m**, wait **2 minutes**, record loss %.
3. Repeat at **15 m** and through **one wall**, then **two walls**.
4. Keep everything else the same: same node, same height, same time.
5. Plot distance vs loss and write one conclusion.

Position	Sent	Loss %
2 m, same room		
15 m, corridor		
1 wall		
2 walls		

Analyst: 60 packets in 2 minutes is a small sample. One lost packet = 1.7%. Is a 2% difference real?

Fair test

Only one variable changes: position. What else could change loss if you are not careful?

Test like an engineer

Troubleshooting

Symptom	Likely cause
Base shows nothing	wrong MAC in BASE[] or base not running; print WiFi.macAddress() again
Compile error in onRecv	core version: 3.x uses esp_now_recv_info_t, 2.x uses const uint8_t *mac
Light reads 0 or 4095 always	AO on an ADC2 pin; move to GPIO 34
Temp shows nan	DHT22 data pin or read faster than every 2 s
Loss looks negative or huge	node rebooted and seq restarted: reset the base too

Debug checklist

1. Same struct on both sides?
2. Same core version?
3. Serial print in the callback?
4. Power bank asleep?

Debug like a network engineer: **first prove the sender sends** (Serial print), then prove the receiver hears. Never debug both ends at once.

MAKE

Level up!

Apply what you learned

Choose your challenge



Two-way: base sends an 'LED on' command back to node 1 when it is too hot (O2).



Relay node: make node 2 forward node 3's packets to reach a far room (O3, mini-mesh).



Encrypt: set a PMK/LMK key so other groups cannot read your packets (O1).



Smarter alerts: show 'WEAK' when loss goes over 10% (O5).

Think like an engineer

Your star network covers one floor. The school has 3 floors and a garden. Which topology, and where do the hubs go?

Boss level

Combine ESP-NOW nodes with one Wi-Fi gateway that uploads all readings to the cloud. Hint: both must use the same Wi-Fi channel.

Objective check

Can I really do it? Prove it.

01	Point to the address, payload and sequence number in our **Msg** struct and say what each is for.	○ ○ ○
02	Why did your base not show the next group's data, even though both used ESP-NOW?	○ ○ ○
03	Draw our network. Name its single point of failure and one situation where mesh would be better.	○ ○ ○
04	What goes wrong if the sender's struct has **value** as an int but the receiver's is a float?	○ ○ ○
05	Node 3 sent seq 50 to 249 and the base got 180. Calculate the loss %. How does the base know a node is offline?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: O5 answer: 200 sent, 20 lost, **10%**. Offline = no packet for 10 s.

Show what you know

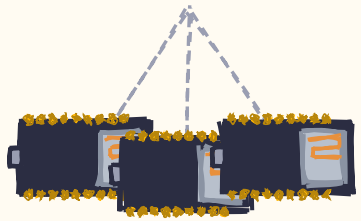
What did we learn?

- ☑ Data travels as **packets**: address + payload + sequence number
- ☑ A **MAC address** sends to one device; broadcast reaches all
- ☑ **Star** is simple but has a single point of failure; **mesh** re-routes
- ☑ Sender and receiver share one **struct**; a **callback** answers each packet
- ☑ Loss % = (sent - received) ÷ sent × 100; silence for 10 s = **OFFLINE**

Exit ticket: Explain to a Grade 5 student why a missing packet is hard to notice without sequence numbers.

At home

Find 3 devices at home that talk wirelessly. Which protocol does each use?



Next class: TL-06 Smart Home Cloud Hub